

Competitive Programming

Section 10 Editorial: Greedy II - Invariants And Constructive Problems

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	4
2	A. Non-Decreasing Repairs	5
2.1	Restatement	5
2.2	Constraints and traps	5
2.3	Observations	5
2.4	Sample walk-through	5
2.5	Algorithm	6
2.6	Correctness proof	6
2.7	Complexity	6
2.8	Full C++ solution	7
2.9	Full Python solution	7
2.10	Common wrong answers	8
2.11	Tests that matter	8
3	B. Bracket Completion	9
3.1	Restatement	9
3.2	Constraints and traps	9
3.3	Observations	9
3.4	Sample walk-through	10
3.5	Algorithm	10
3.6	Correctness proof	11
3.7	Complexity	12
3.8	Full C++ solution	12
3.9	Full Python solution	13
3.10	Common wrong answers	14
3.11	Tests that matter	14
4	C. Bounded Sum Sequence	15
4.1	Restatement	15
4.2	Constraints and traps	15
4.3	Observations	15
4.4	Sample walk-through	16
4.5	Algorithm	16
4.6	Correctness proof	16
4.7	Complexity	17
4.8	Full C++ solution	17
4.9	Full Python solution	18
4.10	Common wrong answers	19
4.11	Tests that matter	19
5	Problem D: Pattern Permutation	20
5.1	Restatement	20
5.2	The forced descending runs	20

5.3	Stack construction and trace	20
5.4	Algorithm	21
5.5	Correctness proof	21
5.6	Complexity and tests	21
5.7	Full C++ solution	22
5.8	Full Python solution	22
6	Stress testing constructive greedy	24
7	What to carry forward	25

1 Editorial overview

The three problems in this section are constructive greedy problems.

The common structure is:

```
build left to right  
keep an invariant  
reject as soon as no completion remains possible
```

For arbitrary-output constructive problems, a normal online judge uses a custom checker. This course's local judge compares exact output, so these exercises ask for canonical answers: minimum cost or lexicographically smallest valid construction.

2 A. Non-Decreasing Repairs

2.1 Restatement

We may increase any array element by 1 per operation. We need the minimum number of operations that makes the array non-decreasing.

2.2 Constraints and traps

The array may be large, and values may be large or negative. The answer can be larger than a 32-bit integer.

Common traps:

- trying to sort the array, which changes element order;
- raising earlier elements, which is impossible because only increases are allowed and the order is fixed;
- using `int` for the total cost.

2.3 Observations

When scanning left to right, suppose the previous repaired value is `need`.

For the current value `x`:

- if `x >= need`, keep it as `x`;
- if `x < need`, it must be raised to at least `need`.

Raising it above `need` cannot help. It only makes later elements need to be larger.

2.4 Sample walk-through

Array:

```
3 2 5 1 7
```

Trace:

```
need = 3
2 < 3, add 1 operation, repaired value is 3
5 >= 3, update need to 5
1 < 5, add 4 operations, repaired value is 5
7 >= 5, update need to 7
```

Answer:

5

2.5 Algorithm

```
cost = 0
need = a[0]

for x in a[1:]:
    if x < need:
        cost += need - x
    else:
        need = x

print cost
```

2.6 Correctness proof

Lemma 1. If the previous repaired value is **need** and the current original value is $x < \text{need}$, every valid repaired array must spend at least $\text{need} - x$ operations on the current element.

Proof. The repaired current value must be at least **need** to preserve the non-decreasing property. Since the value starts at x and each operation increases it by 1, at least $\text{need} - x$ operations are necessary. $\square$

Lemma 2. Raising a current element above **need** is never better than raising it exactly to **need**.

Proof. Raising above **need** costs more immediately. It also increases or keeps the lower bound required for future elements. Therefore it cannot reduce future cost and cannot improve total cost. $\square$

Theorem. The algorithm computes the minimum number of operations.

Proof. At every position, if no repair is needed, the algorithm spends zero operations, which is clearly optimal. If repair is needed, Lemma 1 gives a lower bound of $\text{need} - x$, and the algorithm achieves exactly that lower bound. Lemma 2 shows that spending more cannot help. Summing these forced optimal choices over all positions gives the global minimum. $\square$

2.7 Complexity

```
Time:  O(n)
Memory: O(1) besides input storage
```

2.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

long long solve_case(const vector<long long> &a) {
    long long cost = 0;
    long long need = a[0];
    for (int i = 1; i < (int)a.size(); ++i) {
        if (a[i] < need) {
            cost += need - a[i];
        } else {
            need = a[i];
        }
    }
    return cost;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        vector<long long> a(n);
        for (long long &x : a) cin >> x;
        cout << solve_case(a) << '\n';
    }

    return 0;
}
```

2.9 Full Python solution

```
import sys

def solve_case(a: list[int]) -> int:
    cost = 0
    need = a[0]
    for x in a[1:]:
        if x < need:
            cost += need - x
        else:
            need = x
```

```

    return cost

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        idx += 1
        a = data[idx:idx + n]
        idx += n
        out.append(str(solve_case(a)))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

2.10 Common wrong answers

- Sorting the array and comparing sorted values.
- Updating **need** downward after raising an element.
- Using strict increase instead of non-decreasing.
- Overflowing the total cost in C++.

2.11 Tests that matter

```

already non-decreasing: 1 2 3 -> 0
strictly decreasing:    4 3 2 1 -> 6
large plateau needed:   10 1 1 1 -> 27

```

3 B. Bracket Completion

3.1 Restatement

Replace every '?' in a string with '(' or ')'. The final string must:

1. be a regular bracket sequence;
2. have no strict prefix that is already a regular bracket sequence;
3. be lexicographically smallest among all valid completions.

If no valid completion exists, print **IMPOSSIBLE**.

Because '(' < ')', lexicographically small means using '(' as early as possible.

3.2 Constraints and traps

Common traps:

- accepting ()() even though the prefix () is already balanced;
- forgetting that odd length is impossible;
- filling all early question marks with '(' without checking quotas;
- validating only final balance, not prefix balance.

3.3 Observations

A regular bracket sequence of length n needs exactly:

```
n/2 opening brackets
n/2 closing brackets
```

So the number of '(' and ')' to place is fixed.

To get the lexicographically smallest candidate, fill the earliest question marks with '(' until the opening quota is exhausted. Fill the rest with ')'.
After that, validate:

- balance never negative;
- balance is never zero before the final character;
- final balance is zero.

3.4 Sample walk-through

String:

```
??()
```

The fixed suffix already has one open and one close. We still need one open and one close.

Lexicographically smallest quota fill:

```
(( ))
```

Balance:

```
1 2 1 0
```

This is valid.

String:

```
()??
```

Quota fill:

```
()()
```

Balance:

```
1 0 1 0
```

The strict prefix of length two is balanced, so the answer is IMPOSSIBLE.

3.5 Algorithm

```
if n is odd:
    impossible

need_open = n/2 - count('(')
need_close = n/2 - count(')')

if either quota is negative:
    impossible

for each character:
    if it is '?':
        if need_open > 0:
            put '('
            need_open -= 1
```

```

        else:
            put ')'
            need_close -= 1

balance = 0
for each character index i:
    update balance
    if balance < 0:
        impossible
    if i is not last and balance == 0:
        impossible

if balance == 0:
    print string
else:
    impossible

```

3.6 Correctness proof

Lemma 1. Every valid completion uses exactly $n/2$ opening brackets and $n/2$ closing brackets.

Proof. A regular bracket sequence has final balance zero. Each opening bracket contributes $+1$, and each closing bracket contributes -1 . Therefore the two counts are equal. Since their sum is n , each count is $n/2$. $\square$

Lemma 2. Among all completions satisfying the required bracket counts, the algorithm constructs the lexicographically smallest string.

Proof. The only choice is where to place the remaining opening brackets. Since '(' is smaller than ')', a lexicographically smallest count-respecting string must place opening brackets at the earliest possible question-mark positions until the opening quota is used. That is exactly what the algorithm does. $\square$

Lemma 3. If the algorithm's count-respecting candidate fails the balance validation, no valid completion exists.

Proof. The algorithm places the remaining opening brackets at the earliest possible question-mark positions. Therefore, for every prefix, its number of opening brackets is at least the number of opening brackets in any other count-respecting completion. Equivalently, its balance is at least the balance of any other count-respecting completion at every prefix.

If the algorithm's candidate has negative balance at some prefix, every other candidate has balance no larger there, so every other candidate is also invalid. If the candidate has balance zero at a strict prefix, every other candidate has balance at most zero there, but a valid single-block sequence needs positive balance at every strict prefix. Therefore no other candidate can be valid. $\square$

Theorem. The algorithm prints the required string if it exists under this canonical rule, otherwise IMPOSSIBLE.

Proof. Feasibility of counts is necessary by Lemma 1. The fill step gives the lexicographically smallest count-respecting candidate by Lemma 2. The balance scan exactly checks the two

prefix conditions and final balance. If it passes, the candidate is the answer. If it fails, Lemma 3 applies. $\square$

3.7 Complexity

Time: $O(n)$
Memory: $O(n)$

3.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

string complete(string s) {
    int n = (int)s.size();
    if (n % 2) return "IMPOSSIBLE";

    int need_open = n / 2;
    int need_close = n / 2;
    for (char ch : s) {
        if (ch == '(') --need_open;
        if (ch == ')') --need_close;
    }
    if (need_open < 0 || need_close < 0) return "IMPOSSIBLE";

    for (char &ch : s) {
        if (ch == '?') {
            if (need_open > 0) {
                ch = '(';
                --need_open;
            } else {
                ch = ')';
                --need_close;
            }
        }
    }

    int balance = 0;
    for (int i = 0; i < n; ++i) {
        balance += (s[i] == '(' ? 1 : -1);
        if (balance < 0) return "IMPOSSIBLE";
        if (i + 1 < n && balance == 0) return "IMPOSSIBLE";
    }
    if (balance != 0) return "IMPOSSIBLE";
    return s;
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        string s;
        cin >> s;
        cout << complete(s) << '\n';
    }

    return 0;
}

```

3.9 Full Python solution

```

import sys

def complete(s: str) -> str:
    n = len(s)
    if n % 2:
        return "IMPOSSIBLE"
    need_open = n // 2 - s.count("(")
    need_close = n // 2 - s.count(")")
    if need_open < 0 or need_close < 0:
        return "IMPOSSIBLE"

    chars = list(s)
    for i, ch in enumerate(chars):
        if ch == "?":
            if need_open > 0:
                chars[i] = "("
                need_open -= 1
            else:
                chars[i] = ")"
                need_close -= 1

    balance = 0
    for i, ch in enumerate(chars):
        balance += 1 if ch == "(" else -1
        if balance < 0:
            return "IMPOSSIBLE"
        if i + 1 < n and balance == 0:
            return "IMPOSSIBLE"
    if balance != 0:
        return "IMPOSSIBLE"

```

```

    return "".join(chars)

def main() -> None:
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    t = int(data[0])
    out: list[str] = []
    for i in range(1, t + 1):
        out.append(complete(data[i].decode()))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

3.10 Common wrong answers

- Printing ()() for this task.
- Not rejecting odd length.
- Allowing balance to become negative temporarily.
- Not checking strict prefixes.

3.11 Tests that matter

```

??      -> ()
????    -> (())
()??     -> IMPOSSIBLE
?)       -> ()
)()      -> IMPOSSIBLE

```

4 C. Bounded Sum Sequence

4.1 Restatement

Construct the lexicographically smallest sequence of length n , with every value in $[1, m]$, whose sum is exactly s . If no such sequence exists, print `IMPOSSIBLE`.

4.2 Constraints and traps

The sum can be up to 10^{18} , so use 64-bit integers in C++.

Common traps:

- greedily placing 1 even when the suffix cannot absorb the remaining sum;
- forgetting the lower bound 1 per remaining position;
- printing any valid sequence instead of the lexicographically smallest one.

4.3 Observations

A suffix of length k can realize exactly the sums in:

```
k, k + 1, ..., k*m
```

So before construction, a solution exists exactly when:

```
n <= s <= n*m
```

At a position with `left` positions after it and remaining sum `rem`, choose the smallest x such that the suffix can still realize `rem - x`:

```
left <= rem - x <= left*m
```

The upper-bound side gives:

```
x >= rem - left*m
```

Also $x \geq 1$. Therefore:

```
x = max(1, rem - left*m)
```

This is the smallest feasible value.

4.4 Sample walk-through

For $n = 3$, $m = 5$, $s = 9$:

```
pos 1: left = 2, rem = 9, x = max(1, 9 - 10) = 1
pos 2: left = 1, rem = 8, x = max(1, 8 - 5) = 3
pos 3: left = 0, rem = 5, x = max(1, 5 - 0) = 5
```

Answer:

```
1 3 5
```

4.5 Algorithm

```
if s < n or s > n*m:
    print IMPOSSIBLE

remaining = s
for pos from 0 to n-1:
    left = n - pos - 1
    x = max(1, remaining - left*m)
    append x
    remaining -= x

print sequence
```

4.6 Correctness proof

Lemma 1. A sequence exists if and only if $n \leq s \leq n*m$.

Proof. The minimum possible sum is n , achieved by all ones. The maximum possible sum is $n*m$, achieved by all m . Every integer sum between these values is achievable by starting from all ones and distributing the remaining sum across positions, adding at most $m-1$ to each. $\square$

Lemma 2. At each position, the algorithm chooses the smallest value that allows the remaining suffix to be completed.

Proof. Let $left$ be the number of suffix positions after the current one. After choosing x , the suffix must sum to $remaining - x$. Such a suffix exists exactly when:

```
left <= remaining - x <= left*m
```

The lower-bound side is automatically satisfied by the construction once global feasibility holds and x is not too large. The upper-bound side requires:

```
x >= remaining - left*m
```

Together with $x \geq 1$, the smallest possible choice is:

```
max(1, remaining - left*m)
```

which is exactly the algorithm's choice. $\square$

Lemma 3. After every chosen value, the remaining suffix is feasible.

Proof. By Lemma 2, the chosen value is selected specifically to satisfy the suffix feasibility interval. The remaining sum is therefore between `left` and `left*m`. $\square$

Theorem. The algorithm prints the lexicographically smallest valid sequence.

Proof. Lexicographic order is decided by the first position where two sequences differ. At each position, Lemma 2 shows that the algorithm chooses the smallest value that still permits some valid completion. Any smaller value would make completion impossible. By Lemma 3, the greedy prefix always remains extendable. Therefore no valid sequence can be lexicographically smaller than the algorithm's sequence, and the algorithm's sequence is valid at the end. $\square$

4.7 Complexity

```
Time: O(n)
Memory: O(n) for output
```

4.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

string construct(long long n, long long m, long long s) {
    if (s < n || s > n * m) return "IMPOSSIBLE";

    vector<long long> ans;
    ans.reserve((size_t)n);
    long long remaining = s;
    for (long long pos = 0; pos < n; ++pos) {
        long long left = n - pos - 1;
        long long x = max(1LL, remaining - left * m);
        ans.push_back(x);
        remaining -= x;
    }

    stringstream ss;
    for (int i = 0; i < (int)ans.size(); ++i) {
        if (i) ss << ' ';
        ss << ans[i];
    }
    return ss.str();
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        long long n, m, s;
        cin >> n >> m >> s;
        cout << construct(n, m, s) << '\n';
    }

    return 0;
}

```

4.9 Full Python solution

```

import sys

def construct(n: int, m: int, s: int) -> str:
    if s < n or s > n * m:
        return "IMPOSSIBLE"
    ans: list[int] = []
    remaining = s
    for pos in range(n):
        left = n - pos - 1
        x = max(1, remaining - left * m)
        ans.append(x)
        remaining -= x
    return " ".join(map(str, ans))

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        m = data[idx + 1]
        s = data[idx + 2]
        idx += 3
        out.append(construct(n, m, s))
    sys.stdout.write("\n".join(out))

```

```
if __name__ == "__main__":  
    main()
```

4.10 Common wrong answers

- Always using 1 until the last position, then exceeding m .
- Checking only $s \leq n*m$ and forgetting $s \geq n$.
- Using 32-bit multiplication for $n*m$.
- Producing a valid but not lexicographically smallest sequence.

4.11 Tests that matter

```
n=3, m=5, s=9 -> 1 3 5  
n=4, m=2, s=3 -> IMPOSSIBLE  
n=5, m=10, s=5 -> 1 1 1 1 1  
n=2, m=4, s=8 -> 4 4
```

5 Problem D: Pattern Permutation

5.1 Restatement

Given a pattern of $<$ and $>$ comparisons, construct the lexicographically smallest permutation of $1..n$ satisfying it. A comparison is local, but a run of $>$ creates a global-looking obligation: its values must be strictly decreasing, so we cannot safely print the first value of that run immediately.

For example, the pattern $><<>>$ asks for

```
a[0] > a[1] < a[2] < a[3] > a[4] > a[5].
```

The answer is 2 1 3 6 5 4.

5.2 The forced descending runs

Consider a maximal run of r consecutive $>$ symbols. It involves $r + 1$ positions, and those positions must be strictly decreasing. If the smallest unused labels are $k, k+1, \dots, k+r$, the smallest possible first value of this run is $k+r$: it needs r smaller distinct values after it. Therefore the lexicographically smallest way to fill the run is

```
k+r, k+r-1, ..., k.
```

At a $<$, the run ends. We may now safely print the whole pending descending run and start a new one with the next unused label.

5.3 Stack construction and trace

Read the labels $1, 2, \dots, n$ in increasing order. Push each label onto a stack. When the current comparison is $<$, reverse the pending run by popping the whole stack into the answer. Also flush at the end, because the last run has no later $<$ to trigger it.

For $><<>>$:

push 1, see '>'	pending = [1]	
push 2, see '<'	pop: 2 1	answer = 2 1
push 3, see '<'	pop: 3	answer = 2 1 3
push 4, see '>'	pending = [4]	
push 5, see '>'	pending = [4, 5]	
push 6, end	pop: 6 5 4	answer = 2 1 3 6 5 4

The stack is not searching for a solution. It only delays the output exactly while a $>$ run prevents us from knowing its smallest legal first value.

5.4 Algorithm

```
pending = empty stack
answer = empty list

for value from 1 to n:
    push value onto pending
    if value is n or pattern[value - 1] is '<':
        pop all of pending into answer

print answer
```

5.5 Correctness proof

Lemma 1. The constructed permutation satisfies every comparison.

Proof. Within one flushed stack block, labels are popped in decreasing order, so every comparison inside that block is $>$. A block is flushed only after a $<$ or at the end. Consecutive blocks use consecutive increasing label ranges. The last value of one block is its smallest label, while the first value of the next block is its largest label and is strictly larger. Thus every boundary comparison is $<$. The answer contains every label from 1 through n exactly once, so it is a valid permutation. $\square$

Lemma 2. For any pending $>$ run, the algorithm chooses the smallest possible first value for that run.

Proof. Suppose the run has $r >$ symbols and begins when k is the smallest unused label. Its first value must have r distinct smaller unused values available after it. No value below $k+r$ has enough such values. The algorithm uses $k+r$ first and then $k+r-1, \dots, k$, so it attains that lower bound. $\square$

Theorem. The algorithm outputs the lexicographically smallest valid permutation.

Proof. Process the maximal $>$ runs from left to right. Lemma 2 fixes the smallest possible first value of the current run; once that value is fixed, the same reasoning fixes each later value in the run. A $<$ boundary then forces the next block to use only larger unused labels. By induction over the blocks, no valid permutation can have a smaller value at the first position where it differs from the algorithm's output. $\square$

5.6 Complexity and tests

Every label is pushed and popped once, so time is $O(n)$ and auxiliary memory is $O(n)$. Test a single $<$, a single $>$, an all- $<$ pattern, an all- $>$ pattern, alternating comparisons, and long descending runs at the start or end. The randomized test enumerates all permutations for tiny patterns and checks the lexicographically first valid one.

5.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int tests;
    cin >> tests;
    while (tests--) {
        int n;
        string pattern;
        cin >> n >> pattern;

        vector<int> pending, answer;
        for (int value = 1; value <= n; ++value) {
            pending.push_back(value);
            if (value == n || pattern[value - 1] == '<') {
                while (!pending.empty()) {
                    answer.push_back(pending.back());
                    pending.pop_back();
                }
            }
        }

        for (int i = 0; i < n; ++i) {
            if (i) cout << ' ';
            cout << answer[i];
        }
        cout << '\n';
    }
}
```

5.8 Full Python solution

```
import sys

def construct(pattern: str) -> list[int]:
    pending: list[int] = []
    answer: list[int] = []
    n = len(pattern) + 1

    for value in range(1, n + 1):
        pending.append(value)
        if value == n or pattern[value - 1] == "<":
```

```

        answer.extend(reversed(pending))
        pending.clear()
    return answer

def main() -> None:
    data = sys.stdin.read().split()
    if not data:
        return
    it = iter(data)
    tests = int(next(it))
    out: list[str] = []
    for _ in range(tests):
        n = int(next(it))
        pattern = next(it)
        assert len(pattern) == n - 1
        out.append(" ".join(map(str, construct(pattern))))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

6 Stress testing constructive greedy

Canonical constructive outputs are stress-testable.

For bracket completion:

```
Enumerate every assignment of '?' for  $n \leq 10$ .  
Keep valid strings.  
Compare the greedy output to the smallest valid string.
```

For bounded sums:

```
Enumerate every sequence in  $[1, m]^n$  for small  $n$  and  $m$ .  
Filter by sum.  
Compare to the lexicographically smallest feasible sequence.
```

For non-decreasing repairs, random tests can compare against the direct forced cost formula. A slower dynamic program is possible for tiny bounded values, but the local lower-bound proof is already very direct.

7 What to carry forward

Constructive greedy is about keeping the future alive.

When solving harder constructive problems, ask:

- What must be true about every prefix?
- What resource or quota remains?
- What are the minimum and maximum possibilities for the suffix?
- If I choose the smallest current value, can I still finish?
- If not, what is the smallest value that makes finishing possible?

Those questions appear repeatedly in Codeforces constructive tasks, AtCoder greedy feasibility tasks, and ICPC-style implementation problems.